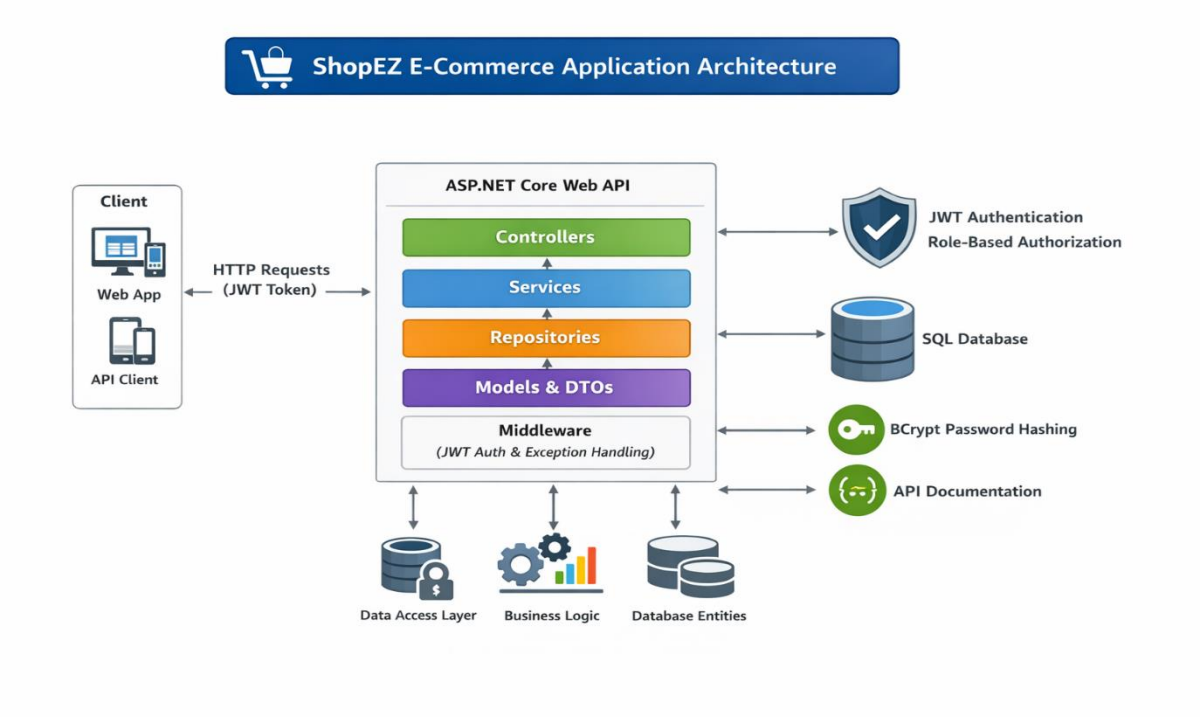
🛒 ShopEZ – E-Commerce Application

A scalable and secure E-Commerce Backend API built using ASP.NET Core Web API.
This project implements authentication, product management, and order processing using JWT-based security and role-based authorization.

Features

- JWT Authentication & Authorization
- User Registration & Login
- Product Management (Admin only)
- Order Management System
- Admin User Management
- Global Exception Handling Middleware
- Swagger API Documentation

Architecture



Architecture Overview

The project follows a **Layered Architecture** for better maintainability and scalability:

- **Controllers** → Handle HTTP requests and responses
- **Services** → Contain business logic

- **Repositories** → Handle database operations
- **Models** → Represent database entities
- **DTOs** → Transfer data between layers
- **Middleware** → Handle global exceptions

Technologies Used

- ASP.NET Core Web AP
- Entity Framework Core
- SQL Server
- JWT Authentication
- JWT Authentication
- Swagger (API Testing)

Authentication & Authorization

JWT Authentication

- Users receive a JWT token after login
- Token must be included in every request

Authorization: Bearer <token>

User Roles

Admin

- Create, update, delete products
- View all orders
- Access all users

Customer

- Place orders
- View own orders only

Project Structure

Controllers/ → API endpoints

Services/ → Business logic

Repositories/ → Data access layer

Models/ → Database entities

DTOs/ → Data transfer objects

Middleware/ → Exception handling

docs/ → Architecture diagrams

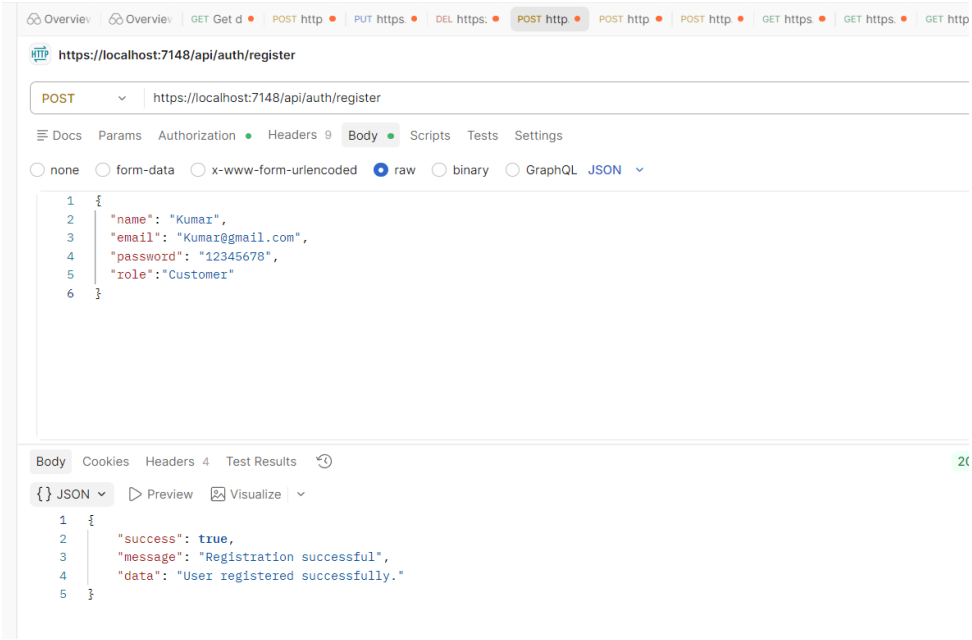
API MODULES

AUTH APIs

Register User

POST : https://localhost:7148 /api/Auth/register

```
{
 "name": "kumar",
 "email": "kumar@gmail.com",
 "password": "12345678",
 "role": "Customer"
}
```

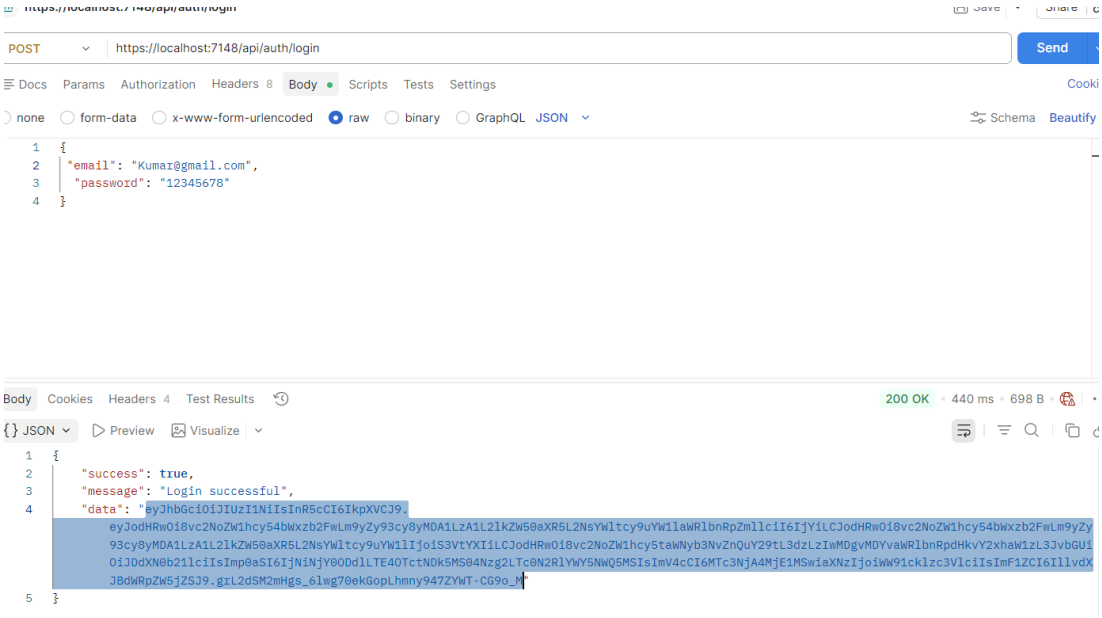


Login User

POST : https://localhost:7148 /api/Auth/login

```
{
 "email": "john@gmail.com",
 "password": "12345678"
}
```

Response → JWT Token

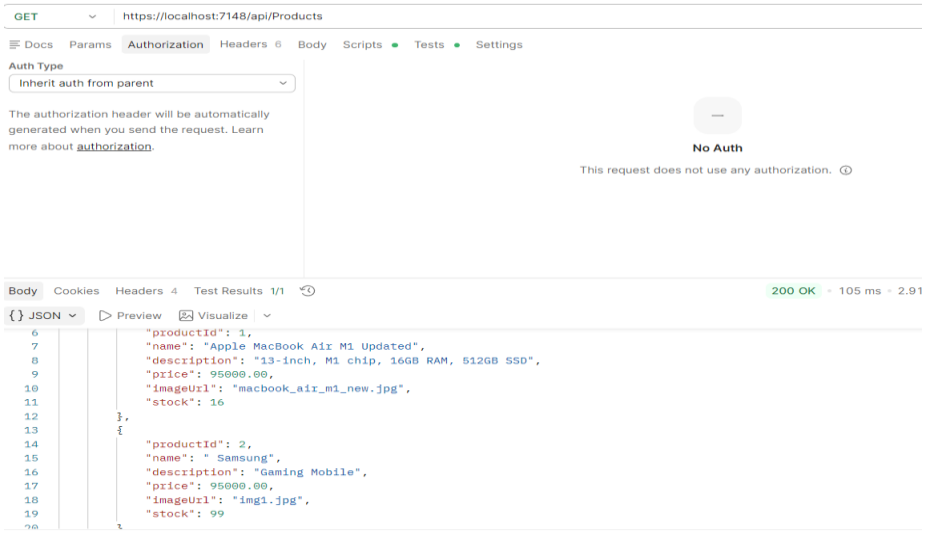


PRODUCT APIs

Method	Endpoint	Access
GET	/api/Products	Public
GET	/api/Products/{id}	Public
POST	/api/Products	Admin
PUT	/api/Products/{id}	Admin
DELETE	/api/Products/{id}	Admin

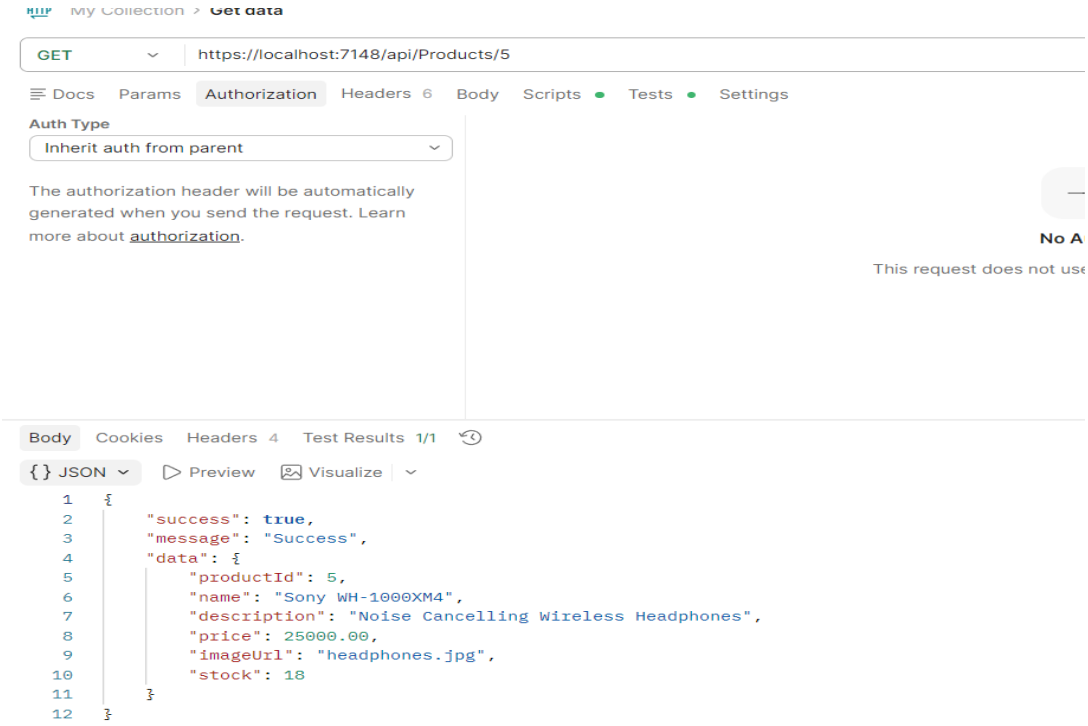
Get All Products

GET: https://localhost:7148 /api/Products



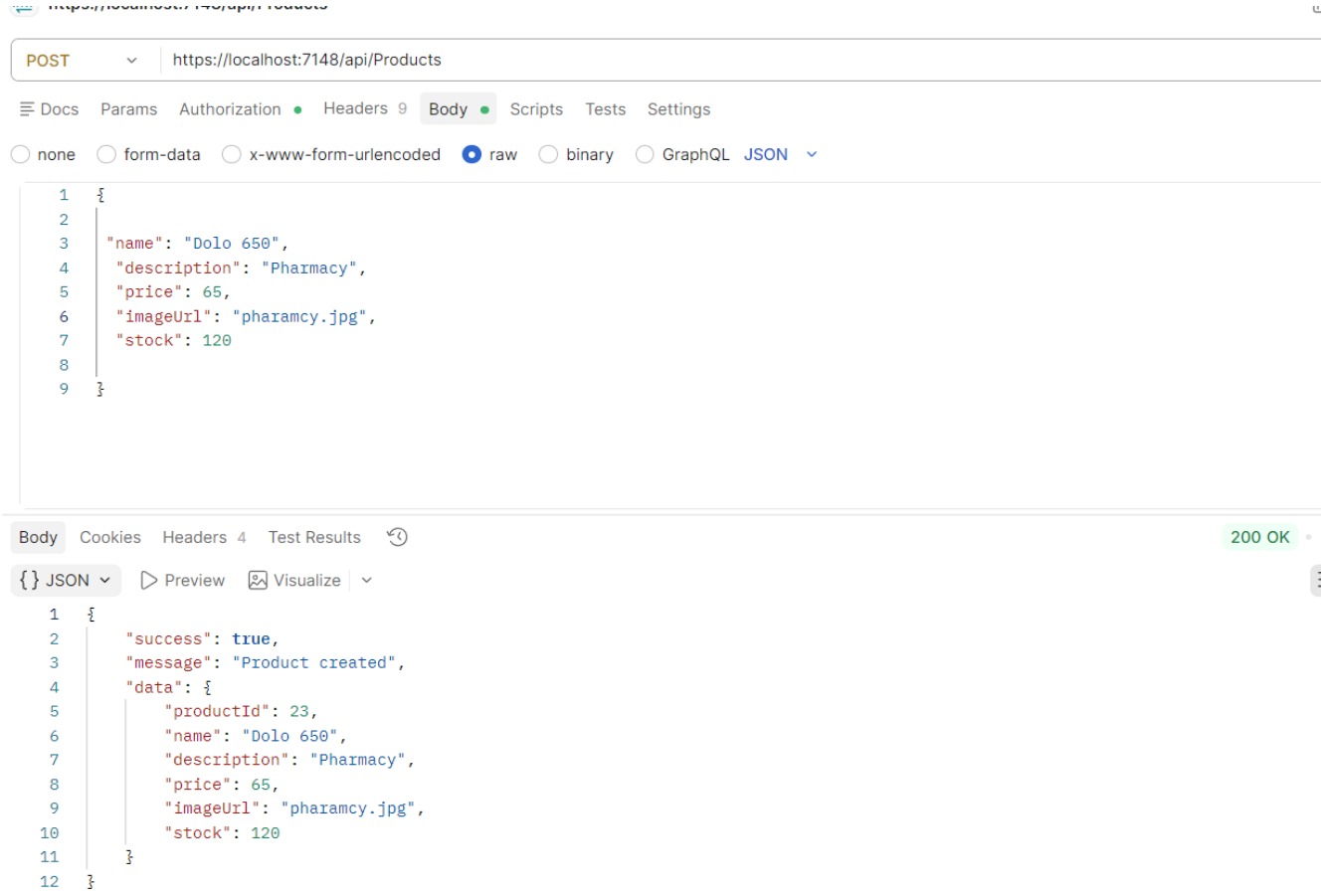
Get Product by ID

GET: https://localhost:7148 /api/Products/{id}



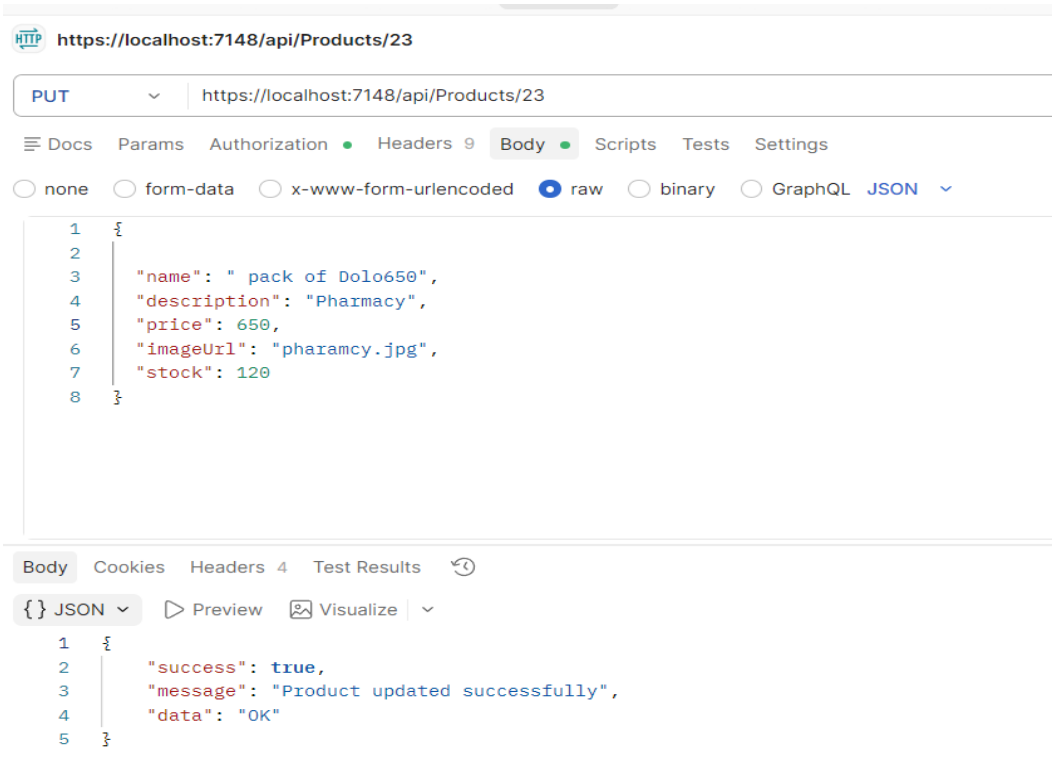
Create Product (Admin)

POST: https://localhost:7148 /api/Products



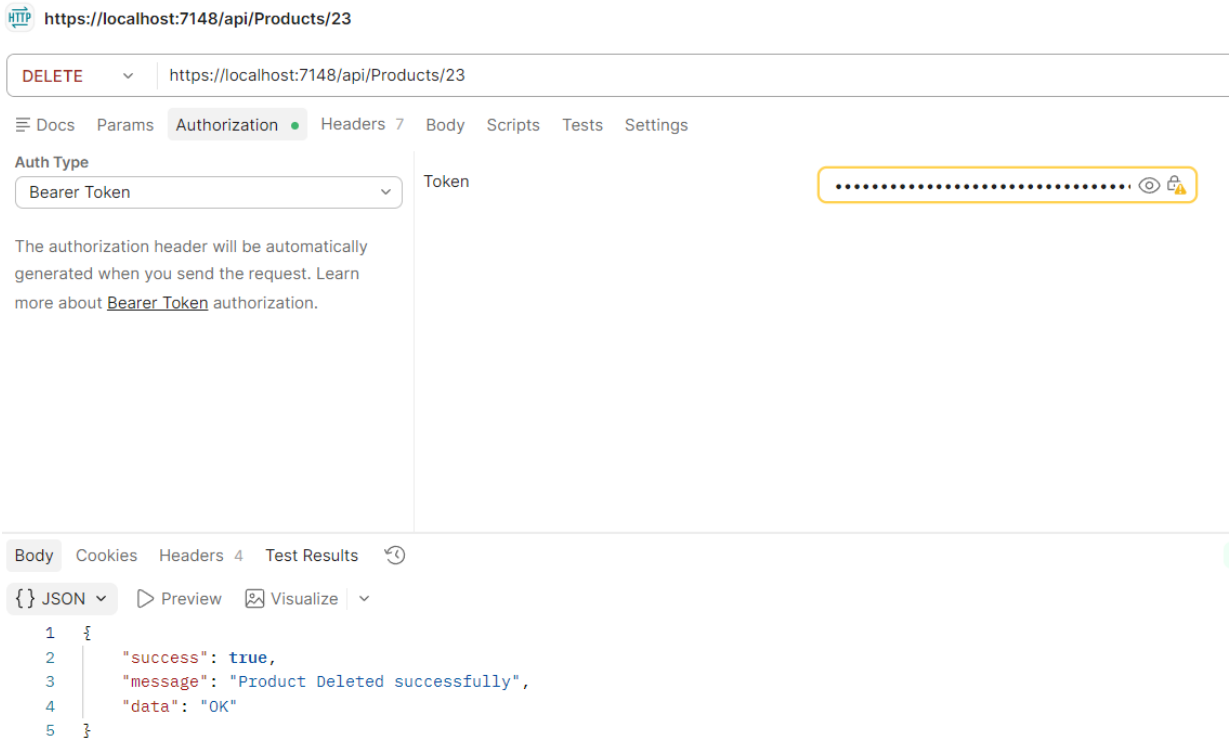
Update Product (Admin)

PUT: https://localhost:7148 /api/Products/{23}



Delete Product (Admin)

DELETE : https://localhost:7148 /api/Products/{id}

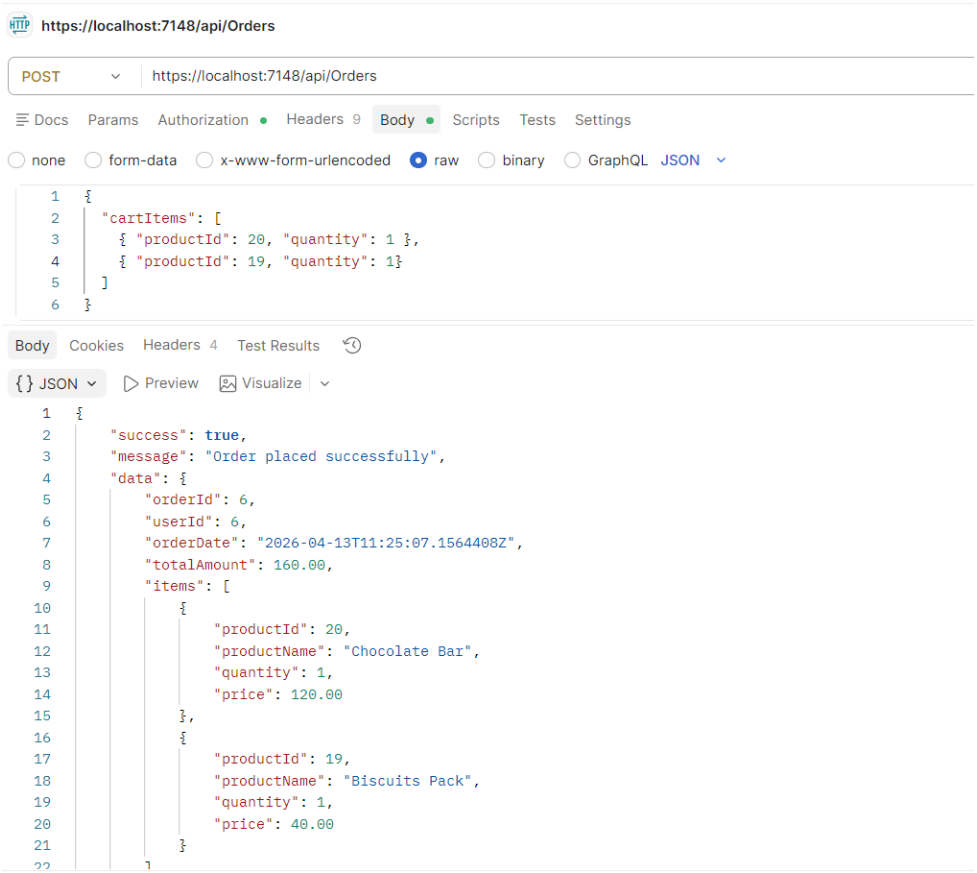


ORDER APIs

Place Order

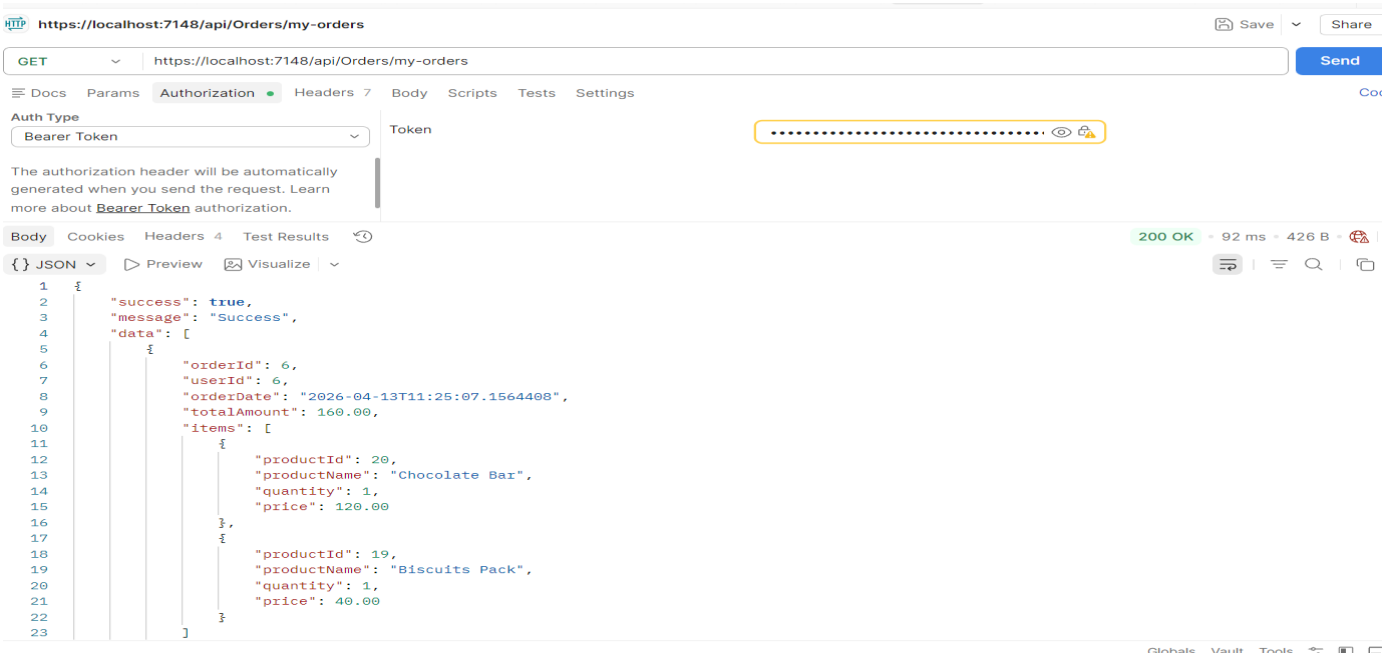
POST: https://localhost:7148/api/Orders

{"cartItems": [{"productId": 1,"quantity": 2}]}



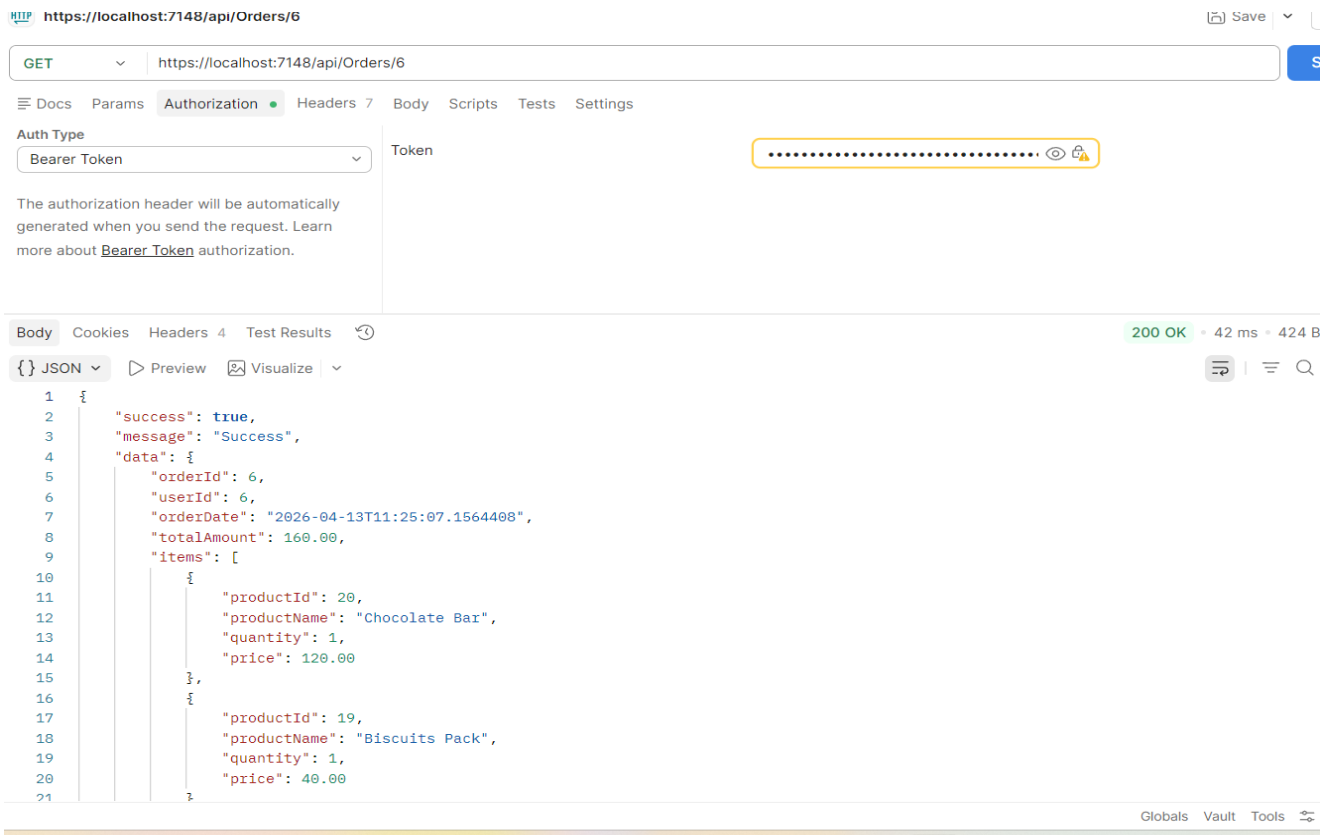
Get My Orders

GET: https://localhost:7148/api/Orders/my-orders



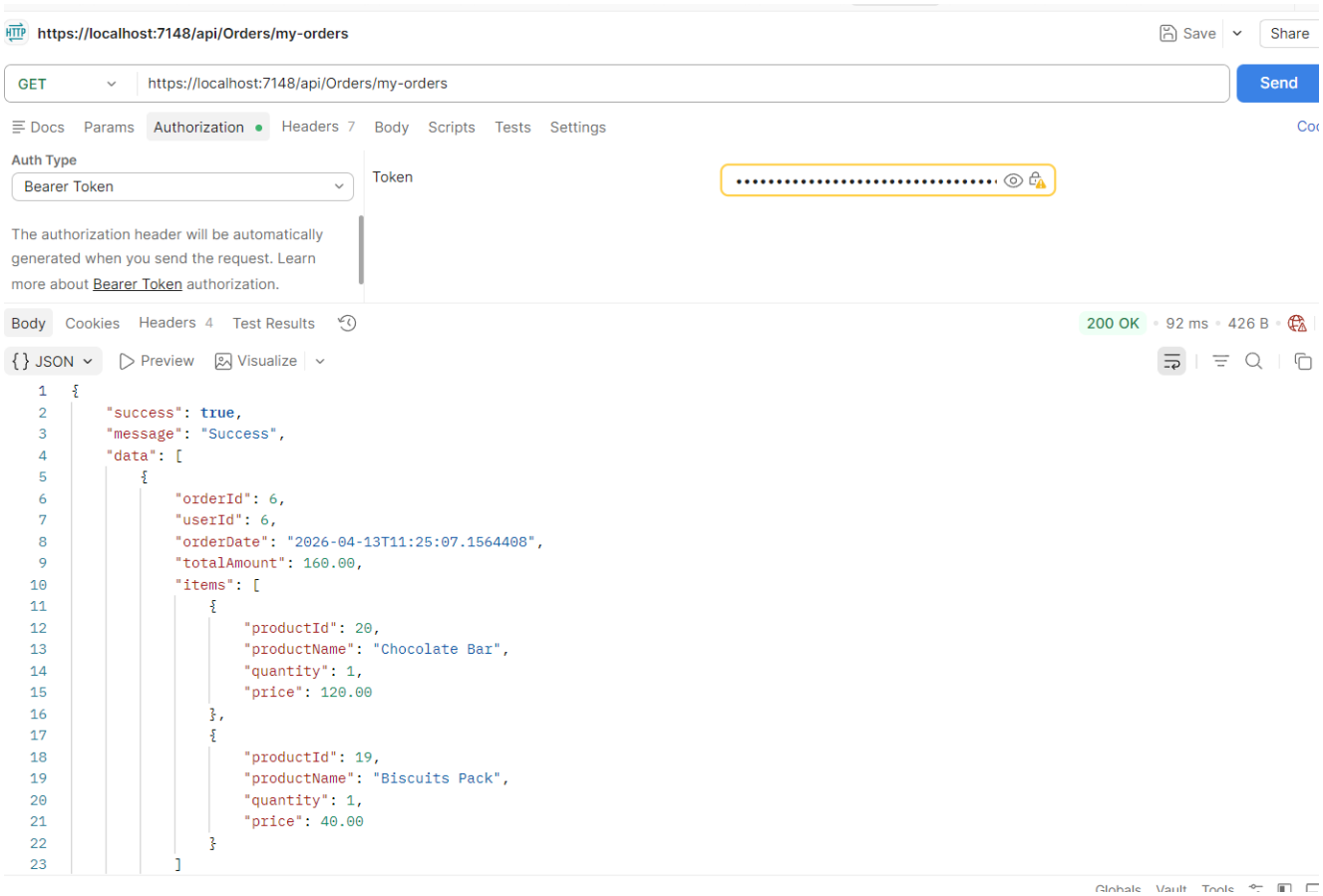
Get Order by ID

GET: https://localhost:7148 /api/Orders/{id}



Get All Orders (Admin)

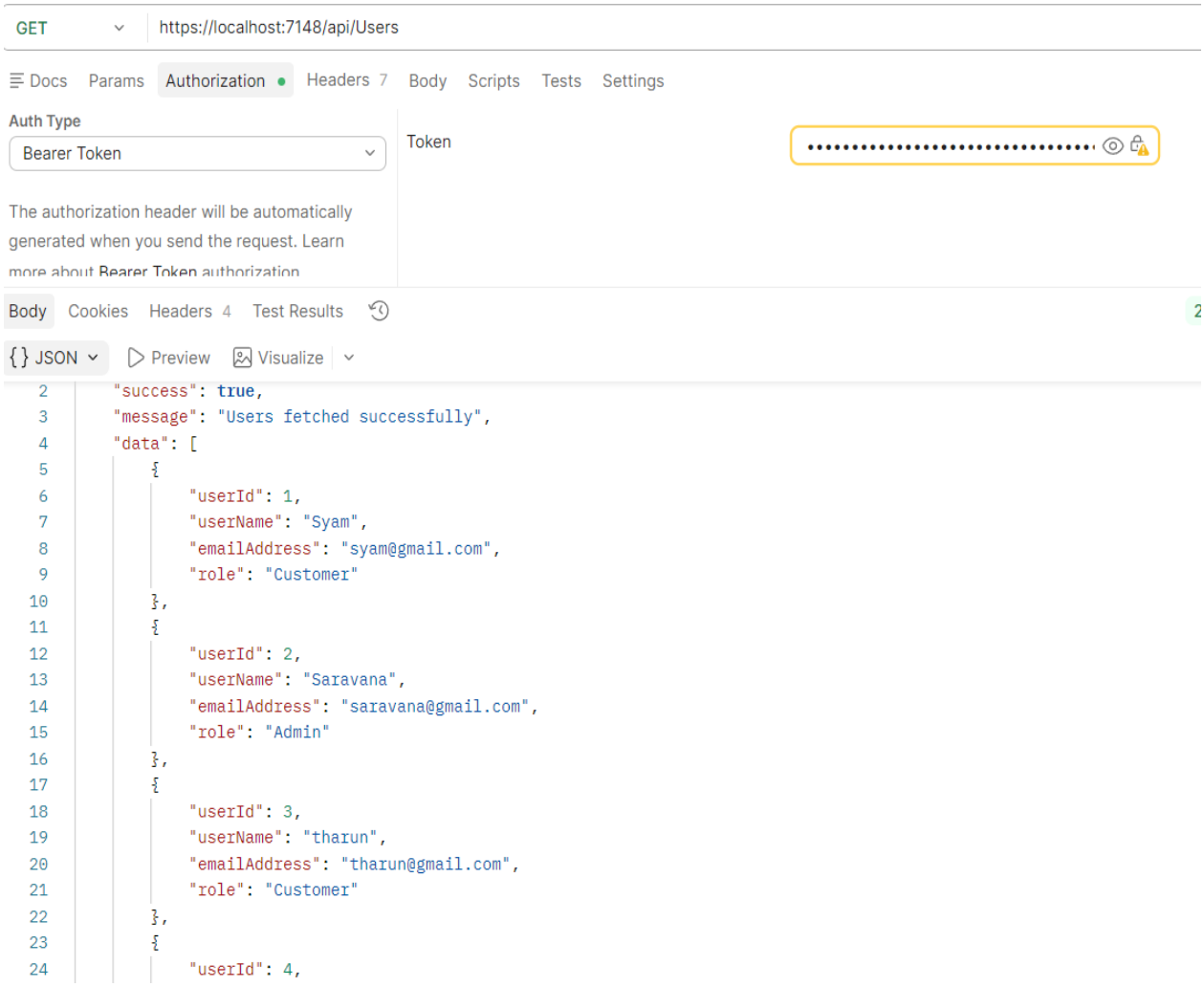
GET: https://localhost:7148/api/Orders/all-orders



USER APIs (Admin Only)

Get All Users

GET: https://localhost:7148/api/Users



API RESPONSE FORMAT

Success Response

```
{
 "success": true,
 "message": "Success",
 "data": {}
}
```

Error Response

```
{
 "success": false,
      "message": "Error message",
      "data": null
}
```

ERROR HANDLING

Handled globally using middleware.

Status Code	Description
400	Bad Request
401	Unauthorized
403	Forbidden
404	Not Found
500	Internal Server Error

SECURITY FEATURES

- Password hashing using BCrypt
- JWT-based authentication
- Role-based authorization
- Protected API endpoints

BUSINESS LOGIC

- Validate product existence before ordering
- Check stock availability
- Reduce stock after order placement
- Restrict users to their own orders
- Allow admin full system access

HOW TO RUN THE PROJECT

1. Open Project

- Open in **Visual Studio 2022**

---

2. Configure Database

Update appsettings.json:

"ConnectionStrings": {

"DefaultConnection": "Your_SQL_Server_Connection"

}

3. Apply Migrations

Add-Migration InitialCreate

Update-Database

---

4. Run Application

dotnet run

---

5. Access Swagger

https://localhost:<port>/swagger

---

6. Test Flow

1.Register user

2.Login → Get JWT Token

3.Authorize in Swagger

4.Test APIs

REQUEST FLOW

Client → Middleware → Authentication → Authorization → Controller → Service → Repository → Database → Response

---

VALIDATION RULES

- Name: Required, Min 3 characters
- Description: Required, Min 5 characters
- Price: Must be greater than 0
- Quantity: Must be ≥ 1
- Email: Must be valid format

FUTURE ENHANCEMENTS

- Payment Gateway Integration
- Order Status Tracking
- Refresh Token Implementation
- Pagination & Filtering
- Email Notifications

CONCLUSION

ShopEZ provides a robust, scalable, and secure backend architecture for an E-Commerce platform using modern ASP.NET Core practices, ensuring maintainability and real-world applicability.

Developed By:

Name: KORUKONDA SYAM SARAVANA KUMAR

Email: syam261004@gmail.com
